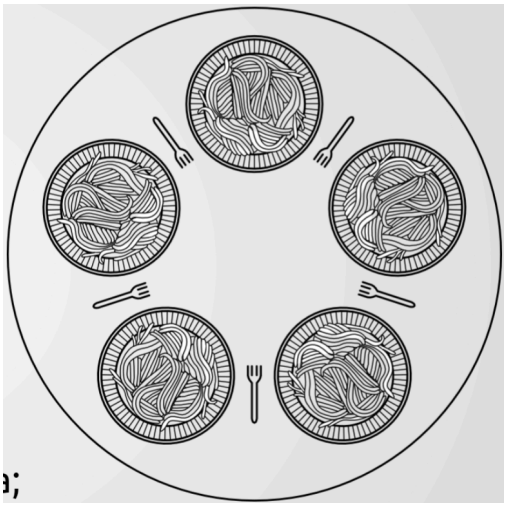
8_9_aprile

Problema dei 5 filosofi

Ciascuno di questi filosofi può mangiare o può pensare, per mangiare richiede due forchette (risorse), le forchette sono una per ogni filosofo e sono tutte condivise. Per risolvere questo problema abbiamo diverse soluzioni (è importante usare 5 filosofi, perché 5 è il numero minimo di filosofi che ci permettono di trovare una soluzione generalizzabile)



Soluzioni generali

1. **Soluzione 1**:

```
int N=5
function philosopher(int i)
     think()
     take_fork(i)
     take_fork((i+1) mod N)
     eat()
     put_fork(i)
     put_fork((i+1) mod N)
```

Questa procedura non è esente da difetti. Se tutti i filosofi diventano affamati nello stesso identico momento e afferrano contemporaneamente la prima forchetta, il sistema si blocca in una situazione di stallo definitivo (*deadlock*), in quanto nessuno di loro avrà mai a disposizione la seconda forchetta necessaria per mangiare.

- **Soluzione 2:** Per evitare lo stallo, si può prevedere che un filosofo, qualora non riesca a prendere la seconda forchetta, rilasci la prima e attenda un tempo prestabilito prima di riprovare. Sebbene eviti il *deadlock*, questa strategia introduce un nuovo problema: se i filosofi agiscono in perfetta sincronia, aspetteranno tutti lo

stesso numero di secondi per poi riattivarsi e bloccarsi di nuovo simultaneamente. Il sistema entra in un ciclo infinito di azioni a vuoto.

- **Soluzione 3:** Questa soluzione è identica alla precedente, ma introduce un tempo di attesa casuale (*random*) anziché fisso. È un approccio semplice ma molto efficace: l'attesa casuale rompe la simmetria e la sincronia tra i filosofi, risolvendo di fatto il problema che creava la soluzione precedente.
- **Soluzione 4:** La soluzione più elegante e robusta prevede l'utilizzo di primitive di sincronizzazione, come un semaforo o un monitor.

Codice usando un semaforo mutex

Di seguito lo pseudocodice di una soluzione usando un semaforo mutex

```
int N=5; int THINKING=0
int HUNGRY=1; int EATING=2
int state[N]
semaphore mutex=1
semaphore s[N]={0,...,0}
```

```
function philosopher(int i)
    while (true) do
        think()
        take_forks(i)
        eat()
        put_forks(i)
```

```
function take_forks(int i)
    down(mutex)
    state[i]=HUNGRY
    test(i)
    up(mutex)
    down(s[i])

function put_forks(int i)
    down(mutex)
    state[i]=THINKING
    test(left(i))
    test(right(i))
    up(mutex)
```

```
function left(int i)  = i-1 mod N
function right(int i) = i+1 mod N

function test(int i)
    if state[i]=HUNGRY and state[left(i)]!=EATING and state[right(i)]!=EATING
        state[i]=EATING
        up(s[i])
```

1. Inizializzazione e Variabili Globali (In alto a sinistra):
   In questa sezione vengono definiti i parametri del sistema:
   - **Stati:** Ogni filosofo può essere in uno di tre stati: `THINKING` (pensa), `HUNGRY` (vorrebbe mangiare) o `EATING` (possiede le forchette e mangia).
   - **Semaforo `mutex`:** Garantisce l'**esclusione mutua** quando si accede o si modifica l'array degli stati.
   - **Semafori `s[N]`:** Un semaforo per ogni filosofo. Vengono usati per "bloccare" un filosofo se le forchette non sono disponibili.
2. Funzione `philosopher` (Al centro a sinistra):
   Rappresenta il **ciclo di vita infinito** di ogni singolo processo (filosofo). È strutturato in una sequenza logica: il filosofo pensa, cerca di prendere le forchette (`take_forks`), mangia e infine le posa (`put_forks`) per ricominciare il ciclo.
3. Funzioni di Gestione Forchette (A destra)

- `take_forks(i)`: Il filosofo dichiara di avere fame e prova a mangiare chiamando `test(i)`. Se i vicini non stanno mangiando, passa a `EATING`. Altrimenti, si blocca sull'istruzione `down(s[i])` finché un vicino non lo "sveglia".
- `put_forks(i)`: Dopo aver mangiato, il filosofo torna a pensare. Fondamentalmente, chiama `test` sui suoi vicini (`left` e `right`) per vedere se la sua azione di posare le forchette permette a uno di loro (che era in attesa) di iniziare a mangiare.

4. Logica di Controllo e Vicinato (In basso)

```
- **`left` / `right`:** Funzioni matematiche (usando l'operatore modulo N)
per identificare correttamente i vicini in un tavolo circolare.
- **`test(i)`:** È il cuore della logica. Verifica se il filosofo i è
affamato e se **entrambi** i suoi vicini non stanno mangiando. Se queste
condizioni sono vere, lo stato diventa `EATING` e viene inviato un segnale
(`up`) al semaforo del filosofo per sbloccarlo.
```

Codice usando un monitor

```
int N=5; int THINKING=0; int HUNGRY=1; int EATING=2

monitor dp_monitor
    int state[N]
    condition self[N]

    function take_forks(int i)
        state[i] = HUNGRY
        test(i)
        if state[i] != EATING
            wait(self[i])

    function put_forks(int i)
        state[i] = THINKING;
        test(left(i));
        test(right(i));

    function test(int i)
        if ( state[left(i)] != EATING and state[i] = HUNGRY
        and state[right(i)] != EATING )
            state[i] = EATING
            signal(self[i])

function philosopher(int i)
    while (true) do
        think()
        dp_monitor.take_forks(i)
        eat()
        dp_monitor.put_forks(i)
```

Questa è un implementazione è molto simile a quella fatta con il semaforo l'unica differenza è che qui sarà il sistema operativo a garantire l'accesso singolo al monitor e quindi alla gestione degli stati

Problema dei lettori-scrittori

Questo problema riguarda processi o thread che devono accedere ad una stessa risorsa (come potrebbe essere una tabella di un database o un'area di memoria): Abbiamo i lettori che hanno solamente bisogno di leggere i dati in quella locazione e gli scrittori che invece sovrascrivono quei dati: si capisce quindi che la sincronizzazione tra questi processi sia fondamentale per evitare problemi.

Soluzione basata sui semafori

```
function reader()
    while true do
        down(mutex)
        rc = rc+1
        if (rc = 1) down(db)
        up(mutex)
        read_database()
        down(mutex)
        rc = rc-1
        if (rc = 0) up(db)
        up(mutex)
        use_data_read()
```

```
semaphore mutex = 1
semaphore db = 1
int rc = 0
```

```
function writer()
    while true do
        think_up_data()
        down(db)
        write_database()
        up(db)
```

Problema

Il problema in questa soluzione è che i lettori hanno priorità assoluta e gli scrittori potrebbero non entrare mai perché mentre un lettore sta già leggendo ne potrebbe entrare un altro aumentando rc di 1 e quindi quando il primo ha finito di leggere rc sarà ancora 1 (perché era 2) e a questo punto il ciclo potrebbe continuare se entra un altro lettore, questo perché i lettori possono entrare in continuazione bloccando la risorsa per gli scrittori praticamente all'infinito

Soluzione basata sui monitor

```
monitor rw_monitor
    int rc = 0; boolean busy_on_write = false
    condition read,write

    function start_read()
        if (busy_on_write OR in_queue(write)) wait(read)
        rc = rc+1
        signal(read)

    function end_read()
        rc = rc-1
        if (rc = 0) signal(write)

    function start_write()
        if (rc > 0 OR busy_on_write) wait(write)
        busy_on_write = true

    function end_write()
        busy_on_write = false
        if (in_queue(write))
            signal(write)
        else
            signal(read)
```

```
function reader()
    while true do
        rw_monitor.start_read()
        read_database()
        rw_monitor.end_read()
        use_data_read()
```

```
function writer()
    while true do
        think_up_data()
        rw_monitor.start_write()
        write_database()
        rw_monitor.end_write()
```

Questa soluzione funziona perché sia lettori che scrittori gestiscono bene l'uso della risorsa senza dare priorità solo ad uno o solo all'altro

Scheduling

La scelta di quali processi va avviato viene fatta dallo scheduler, in pratica sceglie dalla coda dei processi pronti. Quindi lo scheduler entra in gioco alla terminazione o creazione di un processo, quando ci sono degli interrupt o si verificano chiamate bloccanti come l' I/O
Per fare ciò i processi si dividono in due tipi:

- processi *CPU bounded*: un processo che usa principalmente la CPU
- processi *I/O bounded*: un processo che fa principalmente operazioni di input/output quello che fa lo scheduler è inserire tutti i processi I/O bound dentro la CPU in modo da avviare tutte le operazioni di I/O che sono lente, mentre aspetta una risposta avvia i processi CPU bounded, una volta che questi sono finiti riprende l'esecuzione dei processi I/O bounded che a quel punto avranno quasi sicuramente finito

Dispatcher: è l'esecutivo dello scheduler, dopo aver fatto le scelte il dispatcher le mette in atto, il tempo che ci mette il dispatcher per fare le cose si chiama: *latenza di dispatcher*